

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 17 September 2014 - Part I: Theory

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

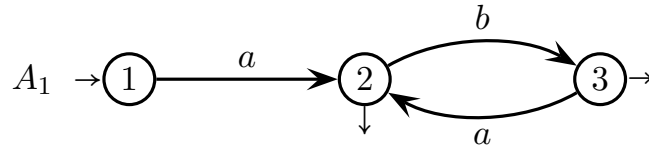
- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression R_1 :

$$R_1 = (a \mid b)^* b$$

and the following deterministic finite automaton A_1 :



which are both defined over the alphabet $\{a, b\}$.

Answer the following questions:

- (a) By means of the Berry-Sethi method (*BS*), find a finite automaton A_2 such that $L(A_2) = L(R_1)$.
 - (b) Find a finite automaton A_3 such that $L(A_3) = L(A_1) \cap L(A_2)$.
 - (c) By means of the node elimination method (Brzozowski), find a regular expression R_2 such that $L(R_2) = L(A_3)$, i.e., the expression R_2 is equivalent to the automaton A_3 found before.
 - (d) (optional) By means of a systematic method of your choice, find a deterministic finite automaton A_4 such that $L(A_4) = \neg L(A_3)$, i.e., the language of A_4 is the complement of that of the automaton A_3 found before.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L over the alphabet with round brackets ‘(’, ‘)’ and square brackets ‘[’, ‘]’.

Answer the following questions:

- (a) Write a *BNF* grammar G' , no matter if ambiguous, that generates the subset of language L the strings of which *do not contain* the digram ‘[(’, i.e., they do not contain an open square bracket immediately followed by an open round bracket.
 - (b) (optional) Write a *BNF* grammar G'' , *not ambiguous*, such that in addition to the previous restriction it has the new restriction of not containing the digram ‘](’, i.e., the strings *do not contain* a closed square bracket immediately followed by an open round bracket.
-

2. The following specifications describe a candidate language for logical programming inspired to the well known Prolog language, although with a few simplifications:

- The program is a list (not empty) of *clauses*, and a clause can be simple or composite.
- A *simple* clause is a predicate terminated by a character ‘.’ (dot).
- A *composite* clause is a predicate, followed by a digram ‘:-’ (colon and dash), followed by a list (not empty) of predicates separated by a character ‘,’ (comma), and terminated by a character ‘.’ (dot).
- A *predicate* is an identifier made of *lowercase* letters, followed by a pair of matching round brackets ‘(’ and ‘)’, which contain a list (not empty) of terms separated by a character ‘,’ (comma).
- A *term* is defined in three possible different ways, as follows:
 - either as an *atom*, i.e., an identifier made of *lowercase* letters
 - or as a *variable*, i.e., an identifier made of *uppercase* letters
 - or as a *functor*, i.e., an identifier made of *lowercase* letters, followed by a pair of matching round brackets ‘(’ and ‘)’, which contain a list (not empty) of terms separated by a character ‘,’ (comma); clearly this last definition of term is *recursive*

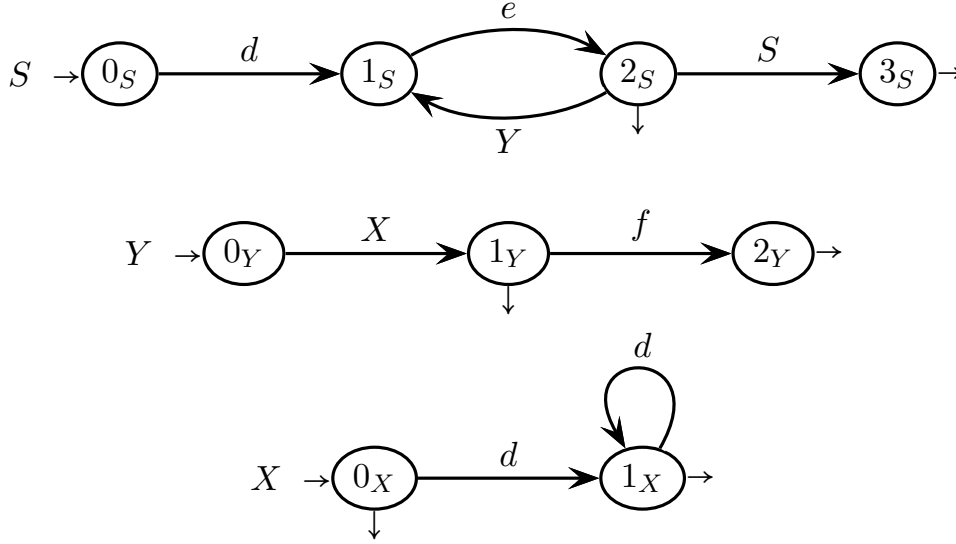
Here is a sample program (the comments do not belong to the language):

```
man (socrates).           /* simple clause (has an atom)      */
int (zero).               /* simple clause (has an atom)      */
dead (X) :- man (X).      /* composite clause (has a variable) */
/* composite clause (has more variables)                       */
friend (X, Y) :- friend (X, Z), friend (Z, Y).
int (succ (X)) :- int (X). /* composite clause (has a functor) */
/* composite clause (has more functors)                       */
equal (add (X, succ (Y)), succ (add (X, Y))) :- int (X), int (Y).
```

Write an extended (*EBNF*) grammar, not ambiguous, that generates the logical programming language described before.

3 Syntax Analysis and Parsing 20%

1. Consider the following recursive network $\mathcal{M}$ of machines, over the terminal alphabet $\{d, e, f\}$ and the nonterminal alphabet $\{S, X, Y\}$ (axiom S):



Answer the following questions:

- (a) Draw the complete pilot $\mathcal{P}$ of the network $\mathcal{M}$, say if the network is of type $ELR(1)$ and explain why.
- (b) Compute all the guide sets on the arcs (shift, call and final darts) of the network $\mathcal{M}$, say if the network is of type $ELL(1)$ and explain why.
- (c) Say if there exists another network $\mathcal{M}'$ equivalent to $\mathcal{M}$ (i.e., that defines the same language $L(\mathcal{M})$), but which has fewer machines; furthermore, if such a network $\mathcal{M}'$ exists, then say which is the minimum number of machines for such a network $\mathcal{M}'$ and explain why.
- (d) (optional) Consider the following valid string (i.e., such that it belongs to language $L(\mathcal{M})$):

$d e f e$

and simulate how to recognize such a string by means of the Earley method; please use the table prepared on the next page. Furthermore, draw the syntax tree of the string recognized.

Earley vector of the valid string $d e f e$ (to be filled)
 (the number of rows is not significant)

0	d	1	e	2	f	3	e	4

syntax tree of the valid string $d e f e$ (to be drawn)

4 Translation and Semantic Analysis 20%

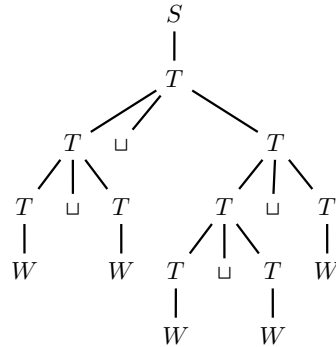
1. A binary string x represents a natural number according to the usual base 2 notation, but with the least significant bit on the left and the most significant bit on the right. For instance, the decimal value 13 (thirteen) is encoded by the binary string $x = 1011$. In this way, one can imagine to input the string x into a translator automaton starting from the least significant bit.

Answer the following questions:

- (a) Draw a sequential (finite) transducer T_1 (sometimes called *sequential machine*) that inputs a binary string x (according to the conventions explained before), with the restriction $|x| \geq 1$ (so the empty string ε is rejected), and that translates it into a binary string y (according to the same conventions) that encodes the numerical value of x incremented by *one*. For instance if the transducer inputs string $x = 1011$, then it outputs string $y = 0111$; and if it inputs $x = 111$, it outputs $y = 0001$ (see the different number of bits of x and y).
 - (b) Draw a sequential (finite) transducer T_2 as before (i.e., with the same conventions and the restriction $|x| \geq 2$), but that increments by *two*. For instance if the transducer inputs string $x = 1011$, then it outputs string $y = 1111$; and if it inputs $x = 111$, it outputs $y = 1001$ (see the different number of bits of x and y).
 - (c) (optional) Write a syntactic translation scheme G_s , G_d (or a translation grammar G_τ) that realizes the translation described at the previous point (b) and is deterministic, and explain why it is correct and deterministic.
-

2. A syntactic support G generates a text (not empty) made of words W (it is not necessary to expand this nonterminal), which are separated by a blank character ‘ $\sqcup$ ’. Here is the support G of the text (axiom S), with a sample syntax tree aside:

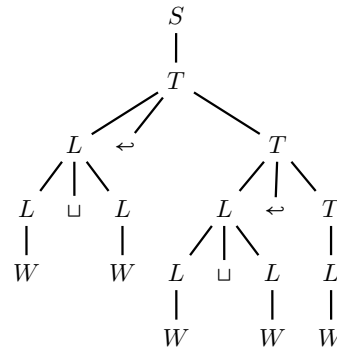
$$G \left\{ \begin{array}{lcl} S & \rightarrow & T \\ T & \rightarrow & T \sqcup T \\ T & \rightarrow & W \end{array} \right.$$



Answer the following questions (use the tables and trees on the next pages):

- (a) By means of an integer attribute $w \geq 1$, write an attribute grammar G_1 that computes how many words there are in the text. The result is expressed by w in the root S . Decorate the sample syntax tree with the values of w .
- (b) By means of the attribute w as before and of a new integer attribute $p \geq 0$, write an attribute grammar G_2 that computes how many words precede each word. The results are expressed by p in the respective leaves W . Decorate the sample syntax tree with the values of w e p .
Furthermore say if the attribute grammar G_2 is of type one-sweep or not, and shortly explain why.
- (c) (optional) The syntactic support G is replaced by a new support G' that can split the text into lines (not empty), as follows (axiom S), where the terminal ' $\leftarrow$ ' represents a newline (there is a sample tree aside):

$$G' \left\{ \begin{array}{l} S \rightarrow T \\ T \rightarrow L \leftrightarrow T \\ T \rightarrow L \\ L \rightarrow L \sqcup L \\ L \rightarrow W \end{array} \right.$$



By means of the attribute w as before and of a new integer attribute $r \geq 0$ (and even more if it helps), write an attribute grammar G_3 (on the support G') that computes how many words there are in the text and how many lines precede each word. The results are expressed by w in the root S and by r in the respective leaves W . Decorate the sample syntax tree with the values of w and r .

Furthermore say if the attribute grammar G_3 is of type one-sweep or not, and shortly explain why.

attributes already assigned to be used for support G

fill the field *type*

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	w	integer ≥ 1	S, T	number of words
	p	integer ≥ 0	T, W	number of words that precede the current word

attributes already assigned to be used for support G'

and possible auxiliary attributes to be added (if it helps)

fill the field *type*

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	w	integer ≥ 1	S, T, L	number of words
	r	integer ≥ 0	T, L, W	number of lines that precede (the line where it is located) the current word

$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow T_1 \sqcup T_2$$

$$3: T_0 \rightarrow W_1$$

$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow T_1 \sqcup T_2$$

$$3: T_0 \rightarrow W_1$$

$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow L_1 \leftrightarrow T_2$$

$$3: T_0 \rightarrow L_1$$

$$4: L_0 \rightarrow L_1 \sqcup L_2$$

$$5: L_0 \rightarrow W_1$$

